

CIFP Carlos III
Modelos Big Data

Sistema Experto de Control Climático de Edificios

Documentación Técnica

UT3T1 - Diseño e implementación de un sistema experto con Expert

Antonio Ferrer Martínez

Marzo 2026

Índice

1. Diseño del sistema	3
1.1 Dominio elegido	3
1.2 Cómo funciona	3
1.3 Hechos del sistema	4
1.4 Cadena de inferencia multinivel	5
2. Reglas y comportamiento	5
2.1 Reglas implementadas	5
2.2 Operadores que he usado	6
3. Análisis de variación	7
3.1 Variación de temperatura	7
3.2 Variación del CO2	7
3.3 Efecto de la ocupación	8
3.4 Efecto de la predicción	8
4. Estrategias de control	8
4.1 Niveles de prioridad	8
4.2 Conflicto confort vs energía	9
5. Series temporales	9
5.1 Datos y modelo	9
5.2 Cómo se integra con el experto	10
6. Plan de pruebas	10
6.1 Casos de prueba	10
6.2 Qué cubre cada grupo	11
7. Anexo	11

1. Diseño del sistema

1.1 Dominio elegido

Para este trabajo elegí hacer un sistema de control climático de un edificio. En la presentación de la UT3 (capítulo 4) se menciona el control industrial como una de las aplicaciones de los sistemas basados en reglas, y un sistema de climatización encaja bien con esa idea: hay datos de sensores, reglas claras de actuación y situaciones donde el orden de las decisiones importa.

El sistema recibe datos de sensores (temperatura, humedad, CO2, ocupación) y decide qué hacer: poner calefacción, refrigeración, ventilación, etc. También tiene una parte de predicción con series temporales para anticiparse a cambios de temperatura.

Encaja bien con Experta porque:

- Los sensores se representan como hechos tipados con `Field`, igual que `Persona` y `Pago` en la sesión 1bis.
- Se puede separar el conocimiento (reglas) de los datos (hechos), que es la base de un sistema experto según la guía técnica (cap. 1).
- El `salience` permite dar prioridad a las emergencias sobre el resto de las reglas.

1.2 Cómo funciona

El sistema sigue el ciclo de ejecución que describe la guía técnica en el capítulo 6 (encadenamiento hacia adelante):

1. `reset()` inicializa la memoria de trabajo y ejecuta los `@DefFacts`, que en mi caso declaran `EstadoSistema(modo="normal")`.

2. Con `declare()` se añaden los hechos de los sensores a la memoria de trabajo.
3. `run()` arranca el ciclo de inferencia: el motor busca qué reglas tienen su LHS (antecedente) satisfecho por los hechos actuales, elige la de mayor `salience`, ejecuta su RHS (consecuente), y repite. Los hechos nuevos que declare el RHS pueden activar más reglas.

1.3 Hechos del sistema

Tengo 11 tipos de hechos. Según la guía técnica (cap. 3), cada `Fact` es una subclase de `dict` que almacena datos etiquetados. Los divido en tres grupos:

Sensores (entrada):

- `SensorTemperatura`: zona, valor, timestamp
- `SensorHumedad`: zona, valor
- `SensorOcupacion`: zona, personas
- `SensorCO2`: zona, ppm
- `PrediccionTemperatura`: zona, valor_futuro, horizonte_min (viene del modelo predictivo)

Hechos derivados (los generan las reglas):

- `NivelConfort`: zona, nivel (frío/óptimo/caluroso)
- `NivelCalidadAire`: zona, nivel (bueno/regular/malo)
- `RiesgoEnergetico`: zona, nivel

Salidas:

- `AccionControl`: zona, sistema, acción, intensidad
- `Alerta`: zona, tipo, mensaje, prioridad
- `EstadoSistema`: modo (normal/ahorro/emergencia)

1.4 Cadena de inferencia multinivel

Las reglas van por niveles, cada nivel usa lo que ha generado el anterior:

```
Nivel 0: Datos de sensores + PrediccionTemperatura
|
| [salience=100] Emergencias
| temp >45 o <0, CO2 >5000 -> Alerta
v
Nivel 1: NivelConfort, NivelCalidadAire
|
| [salience=50] Clasificación
| temp -> frío/óptimo/caluroso
| CO2 -> bueno/regular/malo
v
Nivel 2: RiesgoEnergético
|
| [salience=50] Ocupación alta + temp mala -> riesgo
v
Nivel 3: AccionControl, Alerta
|
| [salience=20] Control
| confort + ocupación -> calef/refrig/ventil
v
Nivel 4: Ajustes
|
| [salience=10] Optimización
| conflicto energía -> bajar intensidad
```

Cuando empecé a hacer las reglas no usaba salience y no me funcionaba porque las reglas de control se ejecutaban antes de que existieran los hechos de clasificación. Separarlo en niveles solucionó el problema.

2. Reglas y comportamiento

2.1 Reglas implementadas

El sistema tiene 17 reglas repartidas en 4 niveles de prioridad.

Emergencias (salience=100) - 2 reglas:

- emergencia_temp: salta si la temperatura pasa de 45 o baja de 0. Usa $P(\text{lambda } t: t > 45 \text{ or } t < 0)$.

- emergencia_co2: salta si el CO2 pasa de 5000 ppm.

Clasificación (salience=50) - 7 reglas:

- 3 para temperatura: frío (<19), óptimo (19-26), caluroso (>26). Llevan NOT(Alerta(...)) para no clasificar si hay emergencia.
- 3 para calidad del aire: bueno (<=400), regular (401-1000), malo (1001-5000).
- 1 de riesgo energético: más de 30 personas y confort no óptimo. Usa OR() para pillar frío o caluroso.

Control (salience=20) - 5 reglas:

- Calefacción si frío + hay gente.
- Refrigeración si caluroso + hay gente.
- Ventilación forzada si aire malo.
- Modo ahorro si no hay nadie y temp óptima.
- Pre-enfriamiento si la predicción dice que va a subir de 28°C (usa series temporales).

Todas llevan NOT() para no duplicar acciones. Al principio no lo ponía y me generaba la misma acción varias veces.

Optimización (salience=10) - 3 reglas:

- Si hay refrigeración alta + riesgo energético, baja a moderada con modify().
- Si caluroso + humedad >70%, activa deshumidificador.
- Ventilación preventiva si hay bastante gente y aire regular.

2.2 Operadores que he usado

- P(lambda): para condiciones numéricas, como P(lambda t: t > 45 or t < 0).

- `L("valor")`: para comparar con un valor exacto.
- `NOT()`: para comprobar que un hecho no existe. Lo uso bastante para evitar duplicados.
- `OR()`: para agrupar condiciones. La uso en la regla de riesgo energético.
- `AS.accion << AccionControl(...)`: captura el hecho para poder modificarlo después con `modify()`.
- `MATCH.zona`: enlaza el campo zona a una variable para usarla luego.

3. Análisis de variación

3.1 Variación de temperatura

Hice un barrido de -5°C a 54°C con CO2 fijo a 500 y 10 personas para ver cómo reacciona el sistema:

Rango	Confort	Acción	Emergencia
$T < 0^{\circ}\text{C}$	no clasifica	parada	SÍ
0-18°C	frío	calefacción	NO
19-26°C	óptimo	ninguna	NO
27-44°C	caluroso	refrigeración	NO
$T > 45^{\circ}\text{C}$	no clasifica	parada	SÍ

Las transiciones salen limpias, no se solapan los rangos. En emergencias no se clasifican porque las reglas de clasificación llevan `NOT(Alerta(...))`.

3.2 Variación del CO2

Rango	Calidad	Ventilación	Emergencia
≤ 400 ppm	bueno	NO	NO
401-1000 ppm	regular	sí >15 personas	NO
1001-5000 ppm	malo	SÍ (forzada)	NO
> 5000 ppm	N/A	SÍ (emergencia)	SÍ

La ventilación preventiva con aire regular es un caso donde se combinan datos de CO2 y de ocupación para decidir.

3.3 Efecto de la ocupación

- 0 personas: modo ahorro (si temp óptima)
- > 0 personas: se activan calefacción/refrigeración
- > 15 personas: ventilación preventiva (con aire regular)
- > 30 personas: riesgo energético (si temp no óptima)

3.4 Efecto de la predicción

Predicción (30 min)	Temp actual	Acción
< 28°C	óptima (22°C)	ninguna
>= 28°C	óptima (22°C)	pre-enfriamiento
>= 28°C	calurosa (30°C)	refrigeración

El pre-enfriamiento solo salta cuando ahora está bien pero se espera que suba. Si ya hace calor, la refrigeración normal tiene más prioridad.

4. Estrategias de control

4.1 Niveles de prioridad

Uso 4 valores de `salience`:

`salience=100` - Emergencias: Lo primero es la seguridad. Si la temperatura es peligrosa o el CO2 es tóxico, se genera una alerta y se bloquean las reglas de clasificación para que no intente climatizar una zona que habría que evacuar.

`salience=50` - Clasificación: Convierte los datos de sensores en categorías (frío, caluroso, aire malo...). Si esto no va antes que el control, las reglas de control no

encuentran los hechos que necesitan y no hacen nada. Es lo que me pasaba al principio.

salience=20 - Control: Genera acciones concretas a partir de las clasificaciones.

salience=10 - Optimización: Ajusta acciones cuando hay conflictos, por ejemplo bajar la intensidad si el gasto es alto.

4.2 Conflicto confort vs energía

Es el caso más interesante. Cuando hace calor y hay mucha gente pasan estas cosas en orden:

1. Se clasifica como caluroso (salience=50)
2. Se detecta riesgo energético porque hay >30 personas con temp no óptima (salience=50)
3. Se activa refrigeración alta (salience=20)
4. La regla de optimización ve que hay refrigeración alta + riesgo energético y baja la intensidad a moderada con `modify()` (salience=10)

Para hacer el `modify()` uso `AS.accion <- AccionControl(...)` para capturar el hecho y luego `self.modify(accion, intensidad="moderada")`. Es parecido al ejemplo de préstamos de la sesión 1bis.

5. Series temporales

5.1 Datos y modelo

Genero datos falsos de temperatura para 7 días (168 registros, uno por hora). Los datos simulan un ciclo día/noche, efecto fin de semana (2 grados menos), tendencia ascendente y ruido aleatorio.

El modelo es una regresión polinomial de grado 3 con scikit-learn. Como features uso:

- `hora_sin` y `hora_cos`: para codificar la hora de forma cíclica (que las 23h estén cerca de las 0h).
- `dia_semana`: número del día.
- `es_finde`: 1 si es sábado o domingo.

5.2 Cómo se integra con el experto

La predicción se mete como un hecho más antes de ejecutar el motor:

```
temp_predicha = predecir_temp(modelo, hora + 0.5, dia)
engine.declare(PrediccionTemperatura(
    zona="oficina",
    valor_futuro=temp_predicha,
    horizonte_min=30
))
```

La regla `control_predictivo` se activa si la predicción pasa de 28°C, la temperatura actual es óptima y hay gente en la zona. Así el sistema puede actuar 30 minutos antes de que suba la temperatura.

6. Plan de pruebas

6.1 Casos de prueba

Hice 12 tests para cubrir distintas situaciones:

ID	Tipo	Prueba	Entrada	Esperado
T01	Normal	Calefacción	Temp=15, Ocup=5	calefacción
T02	Normal	Refrigeración	Temp=32, Ocup=3	refrigeración
T03	Normal	Ventilación	CO2=1500	ventilación
T04	Normal	Sin acción	T=22, CO2=400, Oc=5	óptimo
T05	Multi	Cadena	Temp=15, Ocup=3	frío+calef

ID	Tipo	Prueba	Entrada	Esperado
T06	Multi	Riesgo energ.	T=30, Ocup=50	refrig. mod.
T07	Multi	Predicción	T=24, Pred=30	pre-enfr.
T08	Prior.	Emergencia	Temp=50	alerta crít.
T09	Prior.	Ahorro	Temp=22, Ocup=0	modo ahorro
T10	Límite	Umbral 19	Temp=19.0	óptimo
T11	Robust.	Sin temp	Solo Ocup=5	sin acciones
T12	Robust.	Dos zonas	A:fría, B:caliente	calef+refrig

6.2 Qué cubre cada grupo

- T01-T04: operación normal, que las reglas básicas funcionen.
- T05-T07: inferencia multinivel. T05 comprueba la cadena sensor-confort-acción, T06 que el riesgo modifica la intensidad, T07 que la predicción activa el pre-enfriamiento.
- T08-T09: que el salience funciona bien.
- T10: caso límite, que 19.0 se clasifica como óptimo.
- T11-T12: robustez. T11 que sin datos de temperatura no falla, T12 que dos zonas opuestas se manejan por separado.

Los 12 pasan correctamente.

7. Anexo

A continuación, anexo el cuaderno impreso:

sistema_experto_climatico

March 16, 2026

1 Sistema Experto - Control Climatico de Edificios

UT3T1 - Diseño e implementacion de un sistema experto con Experta

He elegido hacer un sistema que controle la climatizacion de un edificio. La idea es que reciba datos de sensores (temperatura, humedad, CO2 y ocupacion) y decida que hacer: encender calefaccion, refrigeracion, ventilar, etc. Tambien he añadido un modelo de series temporales que predice la temperatura para que el sistema pueda actuar antes de que haga falta.

1.1 Imports

Uso `from experta import *` en vez de `import experta` porque con el `import` normal tendria que escribir `experta.Fact`, `experta.Field`, `experta.Rule`, etc. cada vez. Con el `import *` se importan todos los nombres directamente al namespace y puedo usar `Fact`, `Field`, `Rule` sin el prefijo. Es lo que hace la guia tecnica del profesor en el capitulo 4 y en el notebook de la sesion 1 (el de salud).

El notebook de la sesion 1bis usa `import experta` pero luego escribe `Fact` sin el prefijo, por eso da `NameError` al ejecutarlo. Habria que cambiarlo a `from experta import *` o poner `experta.Fact` en cada sitio.

```
[1]: # !pip install experta scikit-learn matplotlib pandas numpy

# Parche para que experta funcione en Python 3.10+, en versiones nuevas de
# Python quitaron collections.Mapping y experta lo necesita, sacado del issue
# #55 del repo de experta en GitHub: github.com/nlpointer/experta/issues/55
import collections
if not hasattr(collections, 'Mapping'):
    import collections.abc
    collections.Mapping = collections.abc.Mapping

# from import * para no tener que poner experta.Fact, experta.Rule, etc. (ver
# guia tecnica cap. 4)
from experta import *
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
```

```

from sklearn.pipeline import make_pipeline
from datetime import datetime, timedelta
import warnings
warnings.filterwarnings('ignore')

print('Imports OK')

```

Imports OK

1.2 Hechos

Defino los hechos con `Field` para validar los tipos, igual que en el ejemplo de `Persona` y `Pago` de la sesion 1bis. Segun la guia tecnica (cap. 3), un `Fact` es una subclase de `dict` que almacena datos etiquetados, y el motor los guarda en su memoria de trabajo para compararlos con los patrones de las reglas.

Hay 4 sensores como entrada y luego hechos derivados que van creando las reglas durante la inferencia.

[2]: *# Hechos del sistema, como Persona y Pago de la sesion 1bis pero con sensores*

```

class SensorTemperatura(Fact):
    zona = Field(str, mandatory=True)
    valor = Field(float, mandatory=True)          # grados celsius
    timestamp = Field(str, default="2026-03-14T12:00:00")

class SensorHumedad(Fact):
    zona = Field(str, mandatory=True)
    valor = Field(float, mandatory=True)          # porcentaje 0-100

class SensorOcupacion(Fact):
    zona = Field(str, mandatory=True)
    personas = Field(int, mandatory=True)

class SensorCO2(Fact):
    zona = Field(str, mandatory=True)
    ppm = Field(float, mandatory=True)            # partes por millon

class PrediccionTemperatura(Fact):                # viene del modelo de series
    ↪ temporales
    zona = Field(str, mandatory=True)
    valor_futuro = Field(float, mandatory=True)
    horizonte_min = Field(int, default=30)

# Estos los crean las reglas, no los meto yo
class NivelConfort(Fact):                          # frio, optimo o caluroso
    zona = Field(str, mandatory=True)
    nivel = Field(str, mandatory=True)

```

```

class NivelCalidadAire(Fact):
    zona = Field(str, mandatory=True)
    nivel = Field(str, mandatory=True)

class RiesgoEnergetico(Fact):
    zona = Field(str, mandatory=True)
    nivel = Field(str, mandatory=True)

# Salida del sistema
class AccionControl(Fact):
    zona = Field(str, mandatory=True)
    sistema = Field(str, mandatory=True)
    ➔deshumidificador
    accion = Field(str, mandatory=True)
    intensidad = Field(str, default="media")

class Alerta(Fact):
    zona = Field(str, mandatory=True)
    tipo = Field(str, mandatory=True)
    mensaje = Field(str, default="")
    prioridad = Field(str, default="media")

class EstadoSistema(Fact):
    modo = Field(str, default="normal")

print("Hechos definidos")

```

Hechos definidos

1.3 Motor de inferencia

Las reglas se definen como metodos con `@Rule()` dentro de una clase que hereda de `KnowledgeEngine` (guia tecnica, cap. 3). Cada regla tiene una parte LHS (antecedente, los patrones que tienen que coincidir) y una parte RHS (consecuente, lo que se ejecuta).

Las he organizado con **salience** en 4 niveles. Segun la guia tecnica (cap. 6), el motor usa encadenamiento hacia adelante: busca que reglas coinciden con los hechos en memoria, elige la de mayor **salience**, la ejecuta, y repite hasta que no quede ninguna.

- salience 100: emergencias (lo primero es la seguridad)
- salience 50: clasificar datos de sensores en categorias
- salience 20: decidir acciones de control
- salience 10: ajustes y optimizacion

En total son 17 reglas.

```

[3]: # Motor con las 17 reglas separadas por salience (100=emergencias,
    ➔50=clasificar, 20=control, 10=ajustes)

```

```

# Sin el salience no funcionaba porque las de control se ejecutaban antes que
↳ las de clasificacion

class ControlClimatico(KnowledgeEngine):

    @DefFacts()
    def hechos_iniciales(self):
        yield EstadoSistema(modo="normal") # al hacer reset() arranca en normal

    # ---- EMERGENCIAS (salience=100) ----

    @Rule(
        SensorTemperatura(zona=MATCH.zona, valor=P(lambda t: t > 45 or t < 0)),
        ↳ # P() comprueba condicion
        salience=100
    )
    def emergencia_temp(self, zona):
        print(f" EMERGENCIA: Zona '{zona}' - temperatura extrema")
        self.declare(Alerta(zona=zona, tipo="evacuacion",
            mensaje=f"Temperatura extrema en {zona}, evacuar",
            ↳ prioridad="critica"))
        self.declare(AccionControl(zona=zona, sistema="hvac",
            accion="parada_emergencia", intensidad="maxima"))

    @Rule(
        SensorCO2(zona=MATCH.zona, ppm=P(lambda c: c > 5000)),
        salience=100
    )
    def emergencia_co2(self, zona):
        print(f" EMERGENCIA: Zona '{zona}' - CO2 toxico (>5000 ppm)")
        self.declare(Alerta(zona=zona, tipo="evacuacion",
            mensaje=f"CO2 toxico en {zona}, evacuar y ventilar",
            ↳ prioridad="critica"))
        self.declare(AccionControl(zona=zona, sistema="ventilacion",
            accion="ventilacion_emergencia", intensidad="maxima"))

    # ---- CLASIFICACION (salience=50) ----
    # Paso los numeros a categorias. NOT(Alerta critica) para no clasificar en
    ↳ emergencias

    @Rule(
        SensorTemperatura(zona=MATCH.zona, valor=P(lambda t: t < 19)),
        NOT(Alerta(zona=MATCH.zona, prioridad=L("critica"))), # L() compara
        ↳ valor exacto
        salience=50
    )

```

```

def clasificar_frio(self, zona):
    print(f" Clasificacion: Zona '{zona}' -> FRIO")
    self.declare(NivelConfort(zona=zona, nivel="frio"))

@Rule(
    SensorTemperatura(zona=MATCH.zona, valor=P(lambda t: t > 26)),
    NOT(Alerta(zona=MATCH.zona, prioridad=L("critica"))),
    salience=50
)
def clasificar_calor(self, zona):
    print(f" Clasificacion: Zona '{zona}' -> CALUROSO")
    self.declare(NivelConfort(zona=zona, nivel="caluroso"))

@Rule(
    SensorTemperatura(zona=MATCH.zona, valor=P(lambda t: 19 <= t <= 26)),
    NOT(Alerta(zona=MATCH.zona, prioridad=L("critica"))),
    salience=50
)
def clasificar_optimo(self, zona):
    print(f" Clasificacion: Zona '{zona}' -> OPTIMO")
    self.declare(NivelConfort(zona=zona, nivel="optimo"))

@Rule(SensorCO2(zona=MATCH.zona, ppm=P(lambda c: c > 1000 and c <= 5000)),
salience=50)
def clasificar_aire_malo(self, zona):
    print(f" Clasificacion: Zona '{zona}' -> aire MALO")
    self.declare(NivelCalidadAire(zona=zona, nivel="malo"))

@Rule(SensorCO2(zona=MATCH.zona, ppm=P(lambda c: 400 < c <= 1000)),
salience=50)
def clasificar_aire_regular(self, zona):
    print(f" Clasificacion: Zona '{zona}' -> aire REGULAR")
    self.declare(NivelCalidadAire(zona=zona, nivel="regular"))

@Rule(SensorCO2(zona=MATCH.zona, ppm=P(lambda c: c <= 400)), salience=50)
def clasificar_aire_bueno(self, zona):
    print(f" Clasificacion: Zona '{zona}' -> aire BUENO")
    self.declare(NivelCalidadAire(zona=zona, nivel="bueno"))

@Rule(
    SensorOcupacion(zona=MATCH.zona, personas=P(lambda p: p > 30)),
    OR(NivelConfort(zona=MATCH.zona, nivel=L("frio")),      # OR = una u otra
        NivelConfort(zona=MATCH.zona, nivel=L("caluroso"))),
    salience=50
)
def riesgo_energetico(self, zona): # mucha gente + temp no optima = gasto
salto

```



```

    print(f" Clasificacion: Zona '{zona}' -> riesgo energetico ALTO")
    self.declare(RiesgoEnergetico(zona=zona, nivel="alto"))

# ---- CONTROL (salience=20) ----
# NOT(AccionControl...) para que no repita la misma accion, al principio me
↳ pasaba

@Rule(
    NivelConfort(zona=MATCH.zona, nivel=L("frio")),
    SensorOcupacion(zona=MATCH.zona, personas=P(lambda p: p > 0)),
    NOT(AccionControl(zona=MATCH.zona, accion=L("calefaccion"))),
    NOT(Alerta(zona=MATCH.zona, prioridad=L("critica"))),
    salience=20
)
def activar_calefaccion(self, zona): # frio + gente = calefaccion
    print(f" Control: Zona '{zona}' -> activar CALEFACCION")
    self.declare(AccionControl(zona=zona, sistema="hvac",
        accion="calefaccion", intensidad="alta"))

@Rule(
    NivelConfort(zona=MATCH.zona, nivel=L("caluroso")),
    SensorOcupacion(zona=MATCH.zona, personas=P(lambda p: p > 0)),
    NOT(AccionControl(zona=MATCH.zona, accion=L("refrigeracion"))),
    NOT(Alerta(zona=MATCH.zona, prioridad=L("critica"))),
    salience=20
)
def activar_refrigeracion(self, zona): # calor + gente = aire
    print(f" Control: Zona '{zona}' -> activar REFRIGERACION")
    self.declare(AccionControl(zona=zona, sistema="hvac",
        accion="refrigeracion", intensidad="alta"))

@Rule(
    NivelCalidadAire(zona=MATCH.zona, nivel=L("malo")),
    NOT(AccionControl(zona=MATCH.zona, accion=L("ventilacion"))),
    NOT(AccionControl(zona=MATCH.zona, accion=L("ventilacion_emergencia"))),
    salience=20
)
def ventilacion_forzada(self, zona): # aire malo = ventilar
    print(f" Control: Zona '{zona}' -> VENTILACION forzada")
    self.declare(AccionControl(zona=zona, sistema="ventilacion",
        accion="ventilacion", intensidad="alta"))

@Rule(
    NivelConfort(zona=MATCH.zona, nivel=L("optimo")),
    SensorOcupacion(zona=MATCH.zona, personas=L(0)), # L(0) = exactamente 0
    NOT(AccionControl(zona=MATCH.zona)),
    salience=20

```

```

)
def modo_ahorro(self, zona): # nadie + temp bien = ahorrar
    print(f" Control: Zona '{zona}' -> MODO AHORRO (vacía)")
    self.declare(AccionControl(zona=zona, sistema="hvac",
        accion="modo_ahorro", intensidad="baja"))

@Rule(
    PrediccionTemperatura(zona=MATCH.zona, valor_futuro=P(lambda t: t > 28)),
    NivelConfort(zona=MATCH.zona, nivel=L("optimo")), # ahora esta bien
    SensorOcupacion(zona=MATCH.zona, personas=P(lambda p: p > 0)),
    NOT(AccionControl(zona=MATCH.zona, accion=L("pre_enfriamiento"))),
    salience=20
)
def control_predictivo(self, zona): # el modelo dice que sube, enfriar ya
    print(f" Control predictivo: Zona '{zona}' -> PRE-ENFRIAMIENTO")
    self.declare(AccionControl(zona=zona, sistema="hvac",
        accion="pre_enfriamiento", intensidad="media"))

# ---- OPTIMIZACION (salience=10) ----

@Rule(
    AS.accion << AccionControl(zona=MATCH.zona, accion=L("refrigeracion"),
    intensidad=L("alta")),
    RiesgoEnergetico(zona=MATCH.zona, nivel=L("alto")),
    salience=10
)
def conflicto_confort_energia(self, accion, zona): # gasta mucho, bajo
    print(f" Optimizacion: Zona '{zona}' -> bajar refrigeracion a
    MODERADA")
    self.modify(accion, intensidad="moderada") # modify() como en el
    ejemplo de prestamos

@Rule(
    SensorHumedad(zona=MATCH.zona, valor=P(lambda h: h > 70)),
    NivelConfort(zona=MATCH.zona, nivel=L("caluroso")),
    NOT(AccionControl(zona=MATCH.zona, sistema=L("deshumidificador"))),
    salience=10
)
def ajuste_humedad(self, zona): # calor + humedad = agobio
    print(f" Optimizacion: Zona '{zona}' -> activar DESHUMIDIFICADOR")
    self.declare(AccionControl(zona=zona, sistema="deshumidificador",
        accion="deshumidificar", intensidad="alta"))

```

```

@Rule(
    NivelCalidadAire(zona=MATCH.zona, nivel=L("regular")),
    SensorOcupacion(zona=MATCH.zona, personas=P(lambda p: p > 15)),
    NOT(AccionControl(zona=MATCH.zona, accion=L("ventilacion"))),
    NOT(AccionControl(zona=MATCH.zona, accion=L("ventilacion_emergencia"))),
    salience=10
)
def ventilacion_preventiva(self, zona): # aire regular + mucha gente =
    ↪ventilar por si acaso
    print(f" Optimizacion: Zona '{zona}' -> ventilacion preventiva")
    self.declare(AccionControl(zona=zona, sistema="ventilacion",
        accion="ventilacion", intensidad="baja"))

print("Motor definido con 17 reglas")

```

Motor definido con 17 reglas

1.4 Series temporales

Genero datos de temperatura de 7 dias (una lectura por hora) y entreno un modelo para predecir la temperatura futura. Asi el sistema puede pre-enfriar antes de que suba.

```

[4]: # Genero datos falsos de temperatura para 7 dias (1 por hora = 168 datos)
# Simulo un patron realista con ciclo dia/noche, efecto fin de semana, etc.
def generar_datos_temp(dias=7, seed=42):
    np.random.seed(seed) # seed fija para que salga siempre igual
    fechas = []
    temps = []
    inicio = datetime(2026, 3, 7, 0, 0)

    for d in range(dias):
        for h in range(24):
            fecha = inicio + timedelta(days=d, hours=h)
            base = 22.0 # temperatura
            ↪base
            ciclo = 4.0 * np.sin(2 * np.pi * (h - 8) / 24) # sube de dia,
            ↪baja de noche
            finde = -2.0 if fecha.weekday() >= 5 else 0.0 # fines de
            ↪semana baja un poco
            tendencia = 0.3 * d # cada dia sube
            ↪(simula que llega el calor)
            ruido = np.random.normal(0, 1.5) # variacion
            ↪aleatoria
            temps.append(round(base + ciclo + finde + tendencia + ruido, 1))
            fechas.append(fecha)

    return pd.DataFrame({

```

```

        'fecha': fechas, 'hora': [f.hour for f in fechas],
        'dia_semana': [f.weekday() for f in fechas],
        'temperatura': temps, 'zona': 'oficina_norte'
    })

df_temperatura = generar_datos_temp()
print(f'Datos generados: {len(df_temperatura)} registros')
print(f'Temp media: {df_temperatura["temperatura"].mean():.1f}C, min:␣
    ↳{df_temperatura["temperatura"].min():.1f}C, max:␣
    ↳{df_temperatura["temperatura"].max():.1f}C')
df_temperatura.head(10)

```

Datos generados: 168 registros
 Temp media: 22.3C, min: 14.6C, max: 30.1C

```

[4]:
      fecha  hora  dia_semana  temperatura  zona
0 2026-03-07 00:00:00      0          5      17.3  oficina_norte
1 2026-03-07 01:00:00      1          5      15.9  oficina_norte
2 2026-03-07 02:00:00      2          5      17.0  oficina_norte
3 2026-03-07 03:00:00      3          5      18.4  oficina_norte
4 2026-03-07 04:00:00      4          5      16.2  oficina_norte
5 2026-03-07 05:00:00      5          5      16.8  oficina_norte
6 2026-03-07 06:00:00      6          5      20.4  oficina_norte
7 2026-03-07 07:00:00      7          5      20.1  oficina_norte
8 2026-03-07 08:00:00      8          5      19.3  oficina_norte
9 2026-03-07 09:00:00      9          5      21.8  oficina_norte

```

1.4.1 Entrenar el modelo

Uso regresion polinomial de grado 3. Para la hora uso codificacion ciclica (seno y coseno) porque si no el modelo no entiende que las 23h estan cerca de las 0h.

```

[5]: # Modelo de prediccion con regresion polinomial grado 3
      # Uso PolynomialFeatures + LinearRegression de scikit-learn
def entrenar_modelo(df):
    df_feat = df.copy()
    # Codificacion ciclica: paso la hora a seno y coseno para que el modelo
    # entienda que las 23h estan cerca de las 0h (si uso la hora directa, no)
    df_feat['hora_sin'] = np.sin(2 * np.pi * df_feat['hora'] / 24)
    df_feat['hora_cos'] = np.cos(2 * np.pi * df_feat['hora'] / 24)
    df_feat['es_finde'] = (df_feat['dia_semana'] >= 5).astype(int) # 1 si␣
    ↳sabado o domingo

    X = df_feat[['hora_sin', 'hora_cos', 'dia_semana', 'es_finde']].values #␣
    ↳features
    y = df_feat['temperatura'].values # variable objetivo

```

```

    # Pipeline: primero genera combinaciones polinomiales, luego regresion
    ↪ lineal
    modelo = make_pipeline(PolynomialFeatures(degree=3, include_bias=False),
    ↪ LinearRegression())
    modelo.fit(X, y)

    y_pred = modelo.predict(X)
    rmse = np.sqrt(np.mean((y - y_pred) ** 2)) # error cuadrático medio
    print(f"Modelo entrenado - RMSE: {rmse:.2f}C, R2: {modelo.score(X, y):.4f}")
    return modelo

# Funcion para predecir la temperatura a una hora y dia concretos
def predecir_temp(modelo, hora, dia_semana):
    hora_sin = np.sin(2 * np.pi * hora / 24)
    hora_cos = np.cos(2 * np.pi * hora / 24)
    es_finde = 1 if dia_semana >= 5 else 0
    X = np.array([[hora_sin, hora_cos, dia_semana, es_finde]])
    return round(float(modelo.predict(X)[0]), 1)

modelo_temp = entrenar_modelo(df_temperatura)

# Compruebo que las predicciones tienen sentido
print()
print("Predicciones de ejemplo (lunes):")
for h in [8, 12, 14, 18, 22]:
    print(f" {h:02d}:00 -> {predecir_temp(modelo_temp, h, 0)}C")

```

Modelo entrenado - RMSE: 1.33C, R2: 0.8523

Predicciones de ejemplo (lunes):

```

08:00 -> 23.1C
12:00 -> 25.7C
14:00 -> 26.3C
18:00 -> 25.3C
22:00 -> 20.7C

```

```

[6]: # Grafico 1: datos reales vs lo que predice el modelo (para ver si ajusta bien)
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 7))

ax1.plot(df_temperatura['fecha'], df_temperatura['temperatura'], 'b-', alpha=0.
    ↪ 5, linewidth=0.8, label='Datos reales')
df_pred = df_temperatura.copy()
df_pred['hora_sin'] = np.sin(2 * np.pi * df_pred['hora'] / 24)
df_pred['hora_cos'] = np.cos(2 * np.pi * df_pred['hora'] / 24)
df_pred['es_finde'] = (df_pred['dia_semana'] >= 5).astype(int)
X_all = df_pred[['hora_sin', 'hora_cos', 'dia_semana', 'es_finde']].values

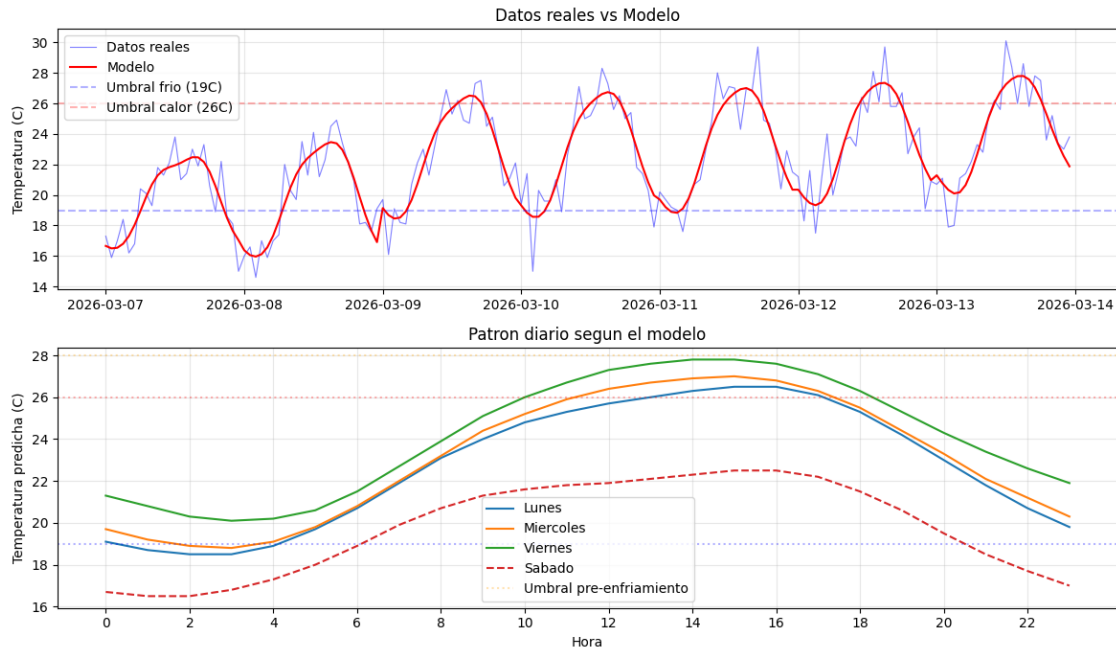
```

```

ax1.plot(df_temperatura['fecha'], modelo_temp.predict(X_all), 'r-', linewidth=1.
↳5, label='Modelo')
ax1.axhline(y=19, color='blue', linestyle='--', alpha=0.3, label='Umbral frio_
↳(19C)')
ax1.axhline(y=26, color='red', linestyle='--', alpha=0.3, label='Umbral calor_
↳(26C)')
ax1.set_ylabel('Temperatura (C)')
ax1.set_title('Datos reales vs Modelo')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Grafico 2: patron por dia de la semana, se ve que el sabado baja
for nombre, dia in [('Lunes', 0), ('Miercoles', 2), ('Viernes', 4), ('Sabado',
↳5)]:
    preds = [predecir_temp(modelo_temp, h, dia) for h in range(24)]
    ax2.plot(range(24), preds, '--' if dia >= 5 else '-', linewidth=1.5,
↳label=nombre)
ax2.axhline(y=19, color='blue', linestyle=':', alpha=0.3)
ax2.axhline(y=26, color='red', linestyle=':', alpha=0.3)
ax2.axhline(y=28, color='orange', linestyle=':', alpha=0.3, label='Umbral_
↳pre-enfriamiento')
ax2.set_xlabel('Hora')
ax2.set_ylabel('Temperatura predicha (C)')
ax2.set_title('Patron diario segun el modelo')
ax2.set_xticks(range(0, 24, 2))
ax2.legend()
ax2.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```



1.5 Ejecucion de escenarios

Funcion auxiliar para no repetir codigo cada vez que pruebo un escenario.

```
[7]: # Funcion auxiliar para probar escenarios sin repetir codigo
# Crea el motor, mete los hechos, ejecuta y muestra los resultados
def ejecutar_escenario(nombre, hechos):
    print("=" * 55)
    print(f"ESCENARIO: {nombre}")
    print("=" * 55)
    engine = ControlClimatico()
    engine.reset() # inicializa memoria de trabajo + DefFacts
    print()
    print("Hechos de entrada:")
    for hecho in hechos:
        engine.declare(hecho) # mete cada hecho en la memoria de trabajo
        tipo = type(hecho).__name__
        campos = {k: v for k, v in hecho.items() if k != '__factid__'}
        print(f"  {tipo}({','.join(f'{k}={v}' for k, v in campos.items())})")
    print()
    print("Reglas disparadas:")
    engine.run() # arranca el ciclo de inferencia
    # Recojo las acciones y alertas que han generado las reglas
    acciones = [h for h in engine.facts.values() if isinstance(h, AccionControl)]
    alertas = [h for h in engine.facts.values() if isinstance(h, Alerta)]
```

```

print()
print(f"Resultado: {len(acciones)} acciones, {len(alertas)} alertas")
for a in acciones:
    print(f"  ACCION: {a['sistema']}/{a['accion']} -> {a['intensidad']}_{a['zona']}")
for a in alertas:
    print(f"  ALERTA [{a['prioridad']}] : {a['mensaje']}")
print()
return engine

```

1.6 Prueba de escenarios

1.6.1 Escenario 1: Oficina fria

Temperatura baja con gente dentro. Tiene que activar calefaccion.

```

[8]: e1 = ejecutar_escenario("Oficina en invierno", [
    SensorTemperatura(zona="oficina_norte", valor=16.5),
    SensorHumedad(zona="oficina_norte", valor=45.0),
    SensorCO2(zona="oficina_norte", ppm=650.0),
    SensorOcupacion(zona="oficina_norte", personas=12),
])

```

```

=====
ESCENARIO: Oficina en invierno
=====

```

Hechos de entrada:

```

+ SensorTemperatura(zona=oficina_norte, valor=16.5)
+ SensorHumedad(zona=oficina_norte, valor=45.0)
+ SensorCO2(zona=oficina_norte, ppm=650.0)
+ SensorOcupacion(zona=oficina_norte, personas=12)

```

Reglas disparadas:

```

Clasificacion: Zona 'oficina_norte' -> aire REGULAR
Clasificacion: Zona 'oficina_norte' -> FRIO
Control: Zona 'oficina_norte' -> activar CALEFACCION

```

Resultado: 1 acciones, 0 alertas

```

ACCION: hvac/calefaccion -> alta (oficina_norte)

```

1.6.2 Escenario 2: Emergencia

Mas de 45 grados, las reglas de emergencia se disparan primero por el salience=100.

```

[9]: e2 = ejecutar_escenario("Emergencia - temperatura extrema", [
    SensorTemperatura(zona="sala_servidores", valor=52.0),
    SensorCO2(zona="sala_servidores", ppm=800.0),
])

```



```
SensorOcupacion(zona="sala_servidores", personas=2),
])
```

```
=====
ESCENARIO: Emergencia - temperatura extrema
=====
```

Hechos de entrada:

```
+ SensorTemperatura(zona=sala_servidores, valor=52.0)
+ SensorCO2(zona=sala_servidores, ppm=800.0)
+ SensorOcupacion(zona=sala_servidores, personas=2)
```

Reglas disparadas:

```
EMERGENCIA: Zona 'sala_servidores' - temperatura extrema
Clasificacion: Zona 'sala_servidores' -> aire REGULAR
```

Resultado: 1 acciones, 1 alertas

```
ACCION: hvac/parada_emergencia -> maxima (sala_servidores)
ALERTA [critica]: Temperatura extrema en sala_servidores, evacuar
```

1.6.3 Escenario 3: Control predictivo

Ahora la temp esta bien (24C) pero el modelo predice que va a subir. Deberia activar pre-enfriamiento.

```
[10]: hora_actual = 12
temp_predicha = predecir_temp(modelo_temp, hora_actual + 0.5, 0)
print(f"El modelo predice {temp_predicha}C para las 12:30")
valor_pred = max(temp_predicha, 29.5) # fuerza que sea alta para que se vea el
    ↪pre-enfriamiento

e3 = ejecutar_escenario("Control predictivo", [
    SensorTemperatura(zona="oficina_sur", valor=24.0), # ahora esta bien
    SensorCO2(zona="oficina_sur", ppm=500.0),
    SensorOcupacion(zona="oficina_sur", personas=8),
    PrediccionTemperatura(zona="oficina_sur", valor_futuro=valor_pred,
    ↪horizonte_min=30),
])
print("Sin la prediccion no se habria activado nada porque 24C esta en rango
    ↪optimo")
```

El modelo predice 25.9C para las 12:30

```
=====
ESCENARIO: Control predictivo
=====
```

Hechos de entrada:

```

+ SensorTemperatura(zona=oficina_sur, valor=24.0)
+ SensorCO2(zona=oficina_sur, ppm=500.0)
+ SensorOcupacion(zona=oficina_sur, personas=8)
+ PrediccionTemperatura(zona=oficina_sur, valor_futuro=29.5, horizonte_min=30)

```

Reglas disparadas:

```

Clasificacion: Zona 'oficina_sur' -> aire REGULAR
Clasificacion: Zona 'oficina_sur' -> OPTIMO
Control predictivo: Zona 'oficina_sur' -> PRE-ENFRIAMIENTO

```

Resultado: 1 acciones, 0 alertas

```

ACCION: hvac/pre_enfriamiento -> media (oficina_sur)

```

Sin la prediccion no se habria activado nada porque 24C esta en rango optimo

1.6.4 Escenario 4: Conflicto confort vs ahorro

Mucha gente + calor genera riesgo energetico. La regla de optimizacion baja la intensidad de la refrigeracion.

```

[11]: e4 = ejecutar_escenario("Conflicto confort vs ahorro", [
    SensorTemperatura(zona="auditorio", valor=30.0),
    SensorHumedad(zona="auditorio", valor=75.0),
    SensorCO2(zona="auditorio", ppm=1200.0),
    SensorOcupacion(zona="auditorio", personas=45),
])

```

```

=====
ESCENARIO: Conflicto confort vs ahorro
=====

```

Hechos de entrada:

```

+ SensorTemperatura(zona=auditorio, valor=30.0)
+ SensorHumedad(zona=auditorio, valor=75.0)
+ SensorCO2(zona=auditorio, ppm=1200.0)
+ SensorOcupacion(zona=auditorio, personas=45)

```

Reglas disparadas:

```

Clasificacion: Zona 'auditorio' -> aire MALO
Clasificacion: Zona 'auditorio' -> CALUROSO
Clasificacion: Zona 'auditorio' -> riesgo energetico ALTO
Control: Zona 'auditorio' -> activar REFRIGERACION
Control: Zona 'auditorio' -> VENTILACION forzada
Optimizacion: Zona 'auditorio' -> bajar refrigeracion a MODERADA
Optimizacion: Zona 'auditorio' -> activar DESHUMIDIFICADOR

```

Resultado: 3 acciones, 0 alertas

```

ACCION: ventilacion/ventilacion -> alta (auditorio)

```

```
ACCION: hvac/refrigeracion -> moderada (auditorio)
ACCION: deshumidificador/deshumidificar -> alta (auditorio)
```

1.6.5 Escenario 5: Dos zonas distintas

Una zona fria y otra caliente. El sistema tiene que manejarlas por separado.

```
[12]: e5 = ejecutar_escenario("Dos zonas independientes", [
    SensorTemperatura(zona="planta_1", valor=15.0),
    SensorCO2(zona="planta_1", ppm=350.0),
    SensorOcupacion(zona="planta_1", personas=10),
    SensorTemperatura(zona="planta_3", valor=29.0),
    SensorCO2(zona="planta_3", ppm=900.0),
    SensorOcupacion(zona="planta_3", personas=8),
])
```

```
=====
ESCENARIO: Dos zonas independientes
=====
```

Hechos de entrada:

```
+ SensorTemperatura(zona=planta_1, valor=15.0)
+ SensorCO2(zona=planta_1, ppm=350.0)
+ SensorOcupacion(zona=planta_1, personas=10)
+ SensorTemperatura(zona=planta_3, valor=29.0)
+ SensorCO2(zona=planta_3, ppm=900.0)
+ SensorOcupacion(zona=planta_3, personas=8)
```

Reglas disparadas:

```
Clasificacion: Zona 'planta_3' -> aire REGULAR
Clasificacion: Zona 'planta_3' -> CALUROSO
Clasificacion: Zona 'planta_1' -> aire BUENO
Clasificacion: Zona 'planta_1' -> FRIO
Control: Zona 'planta_1' -> activar CALEFACCION
Control: Zona 'planta_3' -> activar REFRIGERACION
```

Resultado: 2 acciones, 0 alertas

```
ACCION: hvac/calefaccion -> alta (planta_1)
ACCION: hvac/refrigeracion -> alta (planta_3)
```

1.7 Analisis de sensibilidad

Vario la temperatura y el CO2 para ver como cambian las decisiones del sistema.

```
[13]: # Barrido de temperatura de -5 a 54 para ver como reacciona el sistema
# Con CO2 y ocupacion fijos, asi aislo el efecto de la temperatura
temperaturas = list(range(-5, 55, 1))
```

```

resultados_sens = []
for temp in temperaturas:
    engine = ControlClimatico()
    engine.reset()
    engine.declare(SensorTemperatura(zona="test", valor=float(temp)))
    engine.declare(SensorCO2(zona="test", ppm=500.0))
    engine.declare(SensorOcupacion(zona="test", personas=10))
    engine.run()
    acciones = [h for h in engine.facts.values() if isinstance(h, AccionControl)]
    alertas = [h for h in engine.facts.values() if isinstance(h, Alerta)]
    confort = [h for h in engine.facts.values() if isinstance(h, NivelConfort)]
    resultados_sens.append({
        'temperatura': temp,
        'confort': confort[0]['nivel'] if confort else 'N/A',
        'accion_principal': acciones[0]['accion'] if acciones else 'ninguna',
        'emergencia': any(a['prioridad'] == 'critica' for a in alertas)
    })
df_sens = pd.DataFrame(resultados_sens)

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 6))
colores = {'frio': 'blue', 'optimo': 'green', 'caluroso': 'red', 'N/A': 'gray'}
for nivel, color in colores.items():
    mask = df_sens['confort'] == nivel
    ax1.scatter(df_sens.loc[mask, 'temperatura'], [1]*mask.sum(), c=color,
        label=nivel, s=40)
ax1.axvline(x=0, color='purple', linestyle='--', alpha=0.4)
ax1.axvline(x=19, color='blue', linestyle='--', alpha=0.4)
ax1.axvline(x=26, color='red', linestyle='--', alpha=0.4)
ax1.axvline(x=45, color='purple', linestyle='--', alpha=0.4)
ax1.set_xlabel('Temperatura (C)')
ax1.set_title('Nivel de confort segun temperatura')
ax1.legend()
ax1.set_yticks([])
ax1.grid(True, alpha=0.3)

colores_acc = {'calefaccion': 'blue', 'ninguna': 'lightgray', 'refrigeracion':
    'red', 'parada_emergencia': 'purple'}
for acc in df_sens['accion_principal'].unique():
    mask = df_sens['accion_principal'] == acc
    ax2.scatter(df_sens.loc[mask, 'temperatura'], [1]*mask.sum(), c=colores_acc.get(acc, 'gray'), label=acc, s=40)
ax2.set_xlabel('Temperatura (C)')
ax2.set_title('Accion de control segun temperatura')
ax2.legend()
ax2.set_yticks([])
ax2.grid(True, alpha=0.3)

```

```

plt.tight_layout()
plt.show()

print("\nTransiciones del sistema:")
print("-" * 55)
for (low, high), label in [((-5,-1), 'T < 0C'), ((0,18), '0-18C'),
↳((19,26), '19-26C'), ((27,44), '27-44C'), ((45,54), 'T > 45C')]:
    sub = df_sens[(df_sens['temperatura'] >= low) & (df_sens['temperatura'] <=
↳high)]
    if len(sub) > 0:
        c = sub['confort'].mode().iloc[0]
        a = sub['accion_principal'].mode().iloc[0]
        e = 'SI' if sub['emergencia'].any() else 'NO'
        print(f" {label:>8} -> confort={c}, accion={a}, emergencia={e}")

```

```

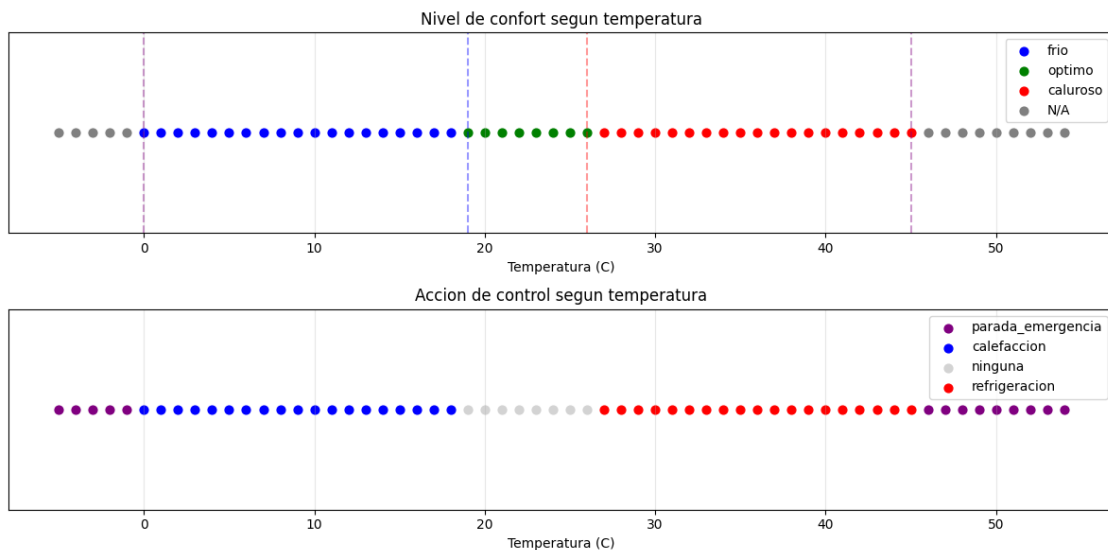
EMERGENCIA: Zona 'test' - temperatura extrema
Clasificacion: Zona 'test' -> aire REGULAR
EMERGENCIA: Zona 'test' - temperatura extrema
Clasificacion: Zona 'test' -> aire REGULAR
EMERGENCIA: Zona 'test' - temperatura extrema
Clasificacion: Zona 'test' -> aire REGULAR
EMERGENCIA: Zona 'test' - temperatura extrema
Clasificacion: Zona 'test' -> aire REGULAR
EMERGENCIA: Zona 'test' - temperatura extrema
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR

```

Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> OPTIMO
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> OPTIMO
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> OPTIMO
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> OPTIMO
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> OPTIMO
Clasificacion: Zona 'test' -> aire REGULAR

[illegible]

Clasificacion: Zona 'test' -> aire REGULAR
 Clasificacion: Zona 'test' -> CALUROSO
 Control: Zona 'test' -> activar REFRIGERACION
 Clasificacion: Zona 'test' -> aire REGULAR
 Clasificacion: Zona 'test' -> CALUROSO
 Control: Zona 'test' -> activar REFRIGERACION
 Clasificacion: Zona 'test' -> aire REGULAR
 Clasificacion: Zona 'test' -> CALUROSO
 Control: Zona 'test' -> activar REFRIGERACION
 Clasificacion: Zona 'test' -> aire REGULAR
 Clasificacion: Zona 'test' -> CALUROSO
 Control: Zona 'test' -> activar REFRIGERACION
 EMERGENCIA: Zona 'test' - temperatura extrema
 Clasificacion: Zona 'test' -> aire REGULAR
 EMERGENCIA: Zona 'test' - temperatura extrema
 Clasificacion: Zona 'test' -> aire REGULAR
 EMERGENCIA: Zona 'test' - temperatura extrema
 Clasificacion: Zona 'test' -> aire REGULAR
 EMERGENCIA: Zona 'test' - temperatura extrema
 Clasificacion: Zona 'test' -> aire REGULAR
 EMERGENCIA: Zona 'test' - temperatura extrema
 Clasificacion: Zona 'test' -> aire REGULAR
 EMERGENCIA: Zona 'test' - temperatura extrema
 Clasificacion: Zona 'test' -> aire REGULAR
 EMERGENCIA: Zona 'test' - temperatura extrema
 Clasificacion: Zona 'test' -> aire REGULAR
 EMERGENCIA: Zona 'test' - temperatura extrema
 Clasificacion: Zona 'test' -> aire REGULAR
 EMERGENCIA: Zona 'test' - temperatura extrema
 Clasificacion: Zona 'test' -> aire REGULAR



Transiciones del sistema:

T < 0C -> confort=N/A, accion=parada_emergencia, emergencia=SI
0-18C -> confort=frio, accion=calefaccion, emergencia=NO
19-26C -> confort=optimo, accion=ninguna, emergencia=NO
27-44C -> confort=caluroso, accion=refrigeracion, emergencia=NO
T > 45C -> confort=N/A, accion=parada_emergencia, emergencia=SI

```
[14]: # Ahora barro el CO2 de 200 a 6000 ppm, con temperatura fija en 22
# para ver solo el efecto del CO2 sin que interfiera la temperatura
niveles_co2 = list(range(200, 6000, 100))
res_co2 = []
for co2 in niveles_co2:
    engine = ControlClimatico()
    engine.reset()
    engine.declare(SensorTemperatura(zona="test", valor=22.0))
    engine.declare(SensorCO2(zona="test", ppm=float(co2)))
    engine.declare(SensorOcupacion(zona="test", personas=10))
    engine.run()
    acciones = [h for h in engine.facts.values() if isinstance(h, AccionControl)]
    alertas = [h for h in engine.facts.values() if isinstance(h, Alerta)]
    calidad = [h for h in engine.facts.values() if isinstance(h, NivelCalidadAire)]
    res_co2.append({
        'co2': co2,
        'calidad_aire': calidad[0]['nivel'] if calidad else 'N/A',
        'ventilacion': any(a['accion'] in ('ventilacion', ventilacion_emergencia) for a in acciones),
        'emergencia': any(a['prioridad'] == 'critica' for a in alertas)
    })
df_co2 = pd.DataFrame(res_co2)

print("Respuesta del sistema segun nivel de CO2:")
print("-" * 50)
for (low, high), label in [((200,400), '<=400 ppm'), ((401,1000), '401-1000 ppm'), ((1001,5000), '1001-5000 ppm'), ((5001,6000), '>5000 ppm')]:
    sub = df_co2[(df_co2['co2'] >= low) & (df_co2['co2'] <= high)]
    if len(sub) > 0:
        cal = sub['calidad_aire'].mode().iloc[0]
        vent = 'SI' if sub['ventilacion'].any() else 'NO'
        emerg = 'SI' if sub['emergencia'].any() else 'NO'
        print(f" {label:>15} -> calidad={cal}, ventilacion={vent}, emergencia={emerg}")
```

Clasificacion: Zona 'test' -> aire BUENO
Clasificacion: Zona 'test' -> OPTIMO
Clasificacion: Zona 'test' -> aire BUENO
Clasificacion: Zona 'test' -> OPTIMO
Clasificacion: Zona 'test' -> aire BUENO
Clasificacion: Zona 'test' -> OPTIMO
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> OPTIMO
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> OPTIMO
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> OPTIMO
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> OPTIMO
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> OPTIMO
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada

Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada

Clasificacion: Zona 'test' -> aire MALO
 Clasificacion: Zona 'test' -> OPTIMO
 Control: Zona 'test' -> VENTILACION forzada
 Clasificacion: Zona 'test' -> aire MALO
 Clasificacion: Zona 'test' -> OPTIMO
 Control: Zona 'test' -> VENTILACION forzada
 Clasificacion: Zona 'test' -> aire MALO
 Clasificacion: Zona 'test' -> OPTIMO
 Control: Zona 'test' -> VENTILACION forzada
 Clasificacion: Zona 'test' -> aire MALO
 Clasificacion: Zona 'test' -> OPTIMO
 Control: Zona 'test' -> VENTILACION forzada
 Clasificacion: Zona 'test' -> aire MALO
 Clasificacion: Zona 'test' -> OPTIMO
 Control: Zona 'test' -> VENTILACION forzada
 Clasificacion: Zona 'test' -> aire MALO
 Clasificacion: Zona 'test' -> OPTIMO
 Control: Zona 'test' -> VENTILACION forzada
 Clasificacion: Zona 'test' -> aire MALO
 Clasificacion: Zona 'test' -> OPTIMO
 Control: Zona 'test' -> VENTILACION forzada
 Clasificacion: Zona 'test' -> aire MALO
 Clasificacion: Zona 'test' -> OPTIMO
 Control: Zona 'test' -> VENTILACION forzada
 Clasificacion: Zona 'test' -> aire MALO
 Clasificacion: Zona 'test' -> OPTIMO
 Control: Zona 'test' -> VENTILACION forzada
 Clasificacion: Zona 'test' -> aire MALO
 Clasificacion: Zona 'test' -> OPTIMO
 Control: Zona 'test' -> VENTILACION forzada
 Clasificacion: Zona 'test' -> aire MALO
 Clasificacion: Zona 'test' -> OPTIMO
 Control: Zona 'test' -> VENTILACION forzada
 EMERGENCIA: Zona 'test' - CO2 toxico (>5000 ppm)
 EMERGENCIA: Zona 'test' - CO2 toxico (>5000 ppm)
 EMERGENCIA: Zona 'test' - CO2 toxico (>5000 ppm)
 EMERGENCIA: Zona 'test' - CO2 toxico (>5000 ppm)
 EMERGENCIA: Zona 'test' - CO2 toxico (>5000 ppm)
 EMERGENCIA: Zona 'test' - CO2 toxico (>5000 ppm)

EMERGENCIA: Zona 'test' - CO2 toxico (>5000 ppm)
EMERGENCIA: Zona 'test' - CO2 toxico (>5000 ppm)
EMERGENCIA: Zona 'test' - CO2 toxico (>5000 ppm)
Respuesta del sistema segun nivel de CO2:

```
-----  
    <=400 ppm -> calidad=bueno, ventilacion=NO, emergencia=NO  
    401-1000 ppm -> calidad=regular, ventilacion=NO, emergencia=NO  
    1001-5000 ppm -> calidad=malo, ventilacion=SI, emergencia=NO  
    >5000 ppm -> calidad=N/A, ventilacion=SI, emergencia=SI
```

1.8 Tests

12 tests que comprueban los distintos comportamientos del sistema.

```
[15]: # 12 tests, cada uno mete datos y comprueba que la salida sea la esperada  
def hacer_test(test_id, nombre, hechos, comprobar):  
    engine = ControlClimatico()  
    engine.reset()  
    for h in hechos:  
        engine.declare(h)  
    engine.run()  
    ok, msg = comprobar(engine)  
    print(f"    [{'PASS' if ok else 'FAIL'}] {test_id}: {nombre}")  
    if not ok: print(f"        -> {msg}")  
    return ok, test_id, nombre  
  
print("Ejecutando tests...")  
print("=" * 55)  
resultados = []  
  
# T01: frio -> calefaccion  
ok, tid, tn = hacer_test("T01", "Temp baja activa calefaccion",  
    [SensorTemperatura(zona="test", valor=15.0), SensorCO2(zona="test", ppm=500.  
    ↪0), SensorOcupacion(zona="test", personas=5)],  
    lambda e: (any(isinstance(h, AccionControl) and h['accion'] ==  
    ↪'calefaccion' for h in e.facts.values()), "No se activo la calefaccion"))  
resultados.append((ok, tid, tn))  
  
# T02: calor -> aire  
ok, tid, tn = hacer_test("T02", "Temp alta activa refrigeracion",  
    [SensorTemperatura(zona="test", valor=32.0), SensorCO2(zona="test", ppm=500.  
    ↪0), SensorOcupacion(zona="test", personas=3)],  
    lambda e: (any(isinstance(h, AccionControl) and h['accion'] ==  
    ↪'refrigeracion' for h in e.facts.values()), "No se activo la refrigeracion"))  
resultados.append((ok, tid, tn))  
  
# T03: CO2 alto -> ventilacion
```

```

ok, tid, tn = hacer_test("T03", "CO2 alto activa ventilacion",
    [SensorTemperatura(zona="test", valor=22.0), SensorCO2(zona="test",
    ↪ppm=1500.0), SensorOcupacion(zona="test", personas=5)],
    lambda e: (any(isinstance(h, AccionControl) and h['accion'] ==
    ↪'ventilacion' for h in e.facts.values()), "No se activo ventilacion"))
resultados.append((ok, tid, tn))

# T04: todo bien -> no hace nada
ok, tid, tn = hacer_test("T04", "Condiciones optimas, sin accion HVAC",
    [SensorTemperatura(zona="test", valor=22.0), SensorCO2(zona="test", ppm=400.
    ↪0), SensorOcupacion(zona="test", personas=5)],
    lambda e: (any(isinstance(h, NivelConfort) and h['nivel'] == 'optimo' for h
    ↪in e.facts.values()) and
        not any(isinstance(h, AccionControl) and h['accion'] in ('calefaccion',
    ↪'refrigeracion') for h in e.facts.values()),
        "Deberia estar en optimo sin acciones de climatizacion"))
resultados.append((ok, tid, tn))

# T05: cadena sensor -> clasifica frio -> calefaccion
ok, tid, tn = hacer_test("T05", "Inferencia multinivel: sensor -> confort ->
    ↪accion",
    [SensorTemperatura(zona="test", valor=15.0), SensorCO2(zona="test", ppm=500.
    ↪0), SensorOcupacion(zona="test", personas=3)],
    lambda e: (any(isinstance(h, NivelConfort) and h['nivel'] == 'frio' for h
    ↪in e.facts.values()) and
        any(isinstance(h, AccionControl) and h['accion'] == 'calefaccion' for h
    ↪in e.facts.values()),
        "Deberia haber NivelConfort(frio) y luego calefaccion"))
resultados.append((ok, tid, tn))

# T06: mucha gente + calor -> refrigeracion moderada (por riesgo energetico)
ok, tid, tn = hacer_test("T06", "Riesgo energetico reduce intensidad",
    [SensorTemperatura(zona="test", valor=30.0), SensorCO2(zona="test", ppm=500.
    ↪0), SensorOcupacion(zona="test", personas=50)],
    lambda e: (any(isinstance(h, RiesgoEnergetico) for h in e.facts.values())
    ↪and
        any(isinstance(h, AccionControl) and h['accion'] == 'refrigeracion' and
    ↪h['intensidad'] == 'moderada' for h in e.facts.values()),
        "Deberia haber riesgo energetico y refrigeracion moderada"))
resultados.append((ok, tid, tn))

# T07: prediccion alta -> pre-enfriamiento
ok, tid, tn = hacer_test("T07", "Prediccion activa pre-enfriamiento",
    [SensorTemperatura(zona="test", valor=24.0), SensorCO2(zona="test", ppm=500.
    ↪0),

```

```

        SensorOcupacion(zona="test", personas=8),
        PrediccionTemperatura(zona="test", valor_futuro=30.0, horizonte_min=30)],
        lambda e: (any(isinstance(h, AccionControl) and h['accion'] ==
        'pre_enfriamiento' for h in e.facts.values()), "No se activo el
        pre-enfriamiento"))
resultados.append((ok, tid, tn))

# T08: 50 grados -> emergencia
ok, tid, tn = hacer_test("T08", "Emergencia tiene prioridad",
    [SensorTemperatura(zona="test", valor=50.0), SensorC02(zona="test", ppm=500.
    0), SensorOcupacion(zona="test", personas=5)],
    lambda e: (any(isinstance(h, Alerta) and h['prioridad'] == 'critica' for h
    in e.facts.values()), "No se genero alerta critica"))
resultados.append((ok, tid, tn))

# T09: nadie + temp bien -> modo ahorro
ok, tid, tn = hacer_test("T09", "Modo ahorro en zona vacia",
    [SensorTemperatura(zona="test", valor=22.0), SensorC02(zona="test", ppm=350.
    0), SensorOcupacion(zona="test", personas=0)],
    lambda e: (any(isinstance(h, AccionControl) and h['accion'] ==
    'modo_ahorro' for h in e.facts.values()), "No se activo modo ahorro"))
resultados.append((ok, tid, tn))

# T10: justo 19.0 = optimo (limite del rango)
ok, tid, tn = hacer_test("T10", "Umbral exacto: 19.0C es optimo",
    [SensorTemperatura(zona="test", valor=19.0), SensorC02(zona="test", ppm=500.
    0), SensorOcupacion(zona="test", personas=5)],
    lambda e: (any(isinstance(h, NivelConfort) and h['nivel'] == 'optimo' for h
    in e.facts.values()), "19.0 deberia ser optimo"))
resultados.append((ok, tid, tn))

# T11: solo ocupacion sin temperatura -> no peta ni hace nada raro
ok, tid, tn = hacer_test("T11", "Sin sensores de temp no hace nada raro",
    [SensorOcupacion(zona="test", personas=5)],
    lambda e: (not any(isinstance(h, AccionControl) for h in e.facts.values())
    and
        not any(isinstance(h, Alerta) for h in e.facts.values()), "No deberia
    generar acciones sin temperatura"))
resultados.append((ok, tid, tn))

# T12: zona A fria + zona B caliente = calefaccion en A y aire en B
ok, tid, tn = hacer_test("T12", "Dos zonas funcionan independientes",
    [SensorTemperatura(zona="zona_A", valor=15.0), SensorC02(zona="zona_A",
    ppm=500.0), SensorOcupacion(zona="zona_A", personas=5),
    SensorTemperatura(zona="zona_B", valor=30.0), SensorC02(zona="zona_B",
    ppm=500.0), SensorOcupacion(zona="zona_B", personas=5)],

```

```

        lambda e: (any(isinstance(h, AccionControl) and h['zona'] == 'zona_A' and
↪h['accion'] == 'calefaccion' for h in e.facts.values()) and
        any(isinstance(h, AccionControl) and h['zona'] == 'zona_B' and
↪h['accion'] == 'refrigeracion' for h in e.facts.values()),
        "Deberia haber calefaccion en A y refrigeracion en B"))
resultados.append((ok, tid, tn))

print("\n" + "=" * 55)
pasados = sum(1 for ok, _, _ in resultados if ok)
print(f"Resultado: {pasados}/{len(resultados)} tests pasados")
for ok, tid, nombre in resultados:
    print(f"  {'PASS' if ok else 'FAIL'} - {tid}: {nombre}")

```

Ejecutando tests...

```

=====
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
[PASS] T01: Temp baja activa calefaccion
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> CALUROSO
Control: Zona 'test' -> activar REFRIGERACION
[PASS] T02: Temp alta activa refrigeracion
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
[PASS] T03: CO2 alto activa ventilacion
Clasificacion: Zona 'test' -> aire BUENO
Clasificacion: Zona 'test' -> OPTIMO
[PASS] T04: Condiciones optimas, sin accion HVAC
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
[PASS] T05: Inferencia multinivel: sensor -> confort -> accion
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> CALUROSO
Clasificacion: Zona 'test' -> riesgo energetico ALTO
Control: Zona 'test' -> activar REFRIGERACION
Optimizacion: Zona 'test' -> bajar refrigeracion a MODERADA
Optimizacion: Zona 'test' -> ventilacion preventiva
[PASS] T06: Riesgo energetico reduce intensidad
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> OPTIMO
Control predictivo: Zona 'test' -> PRE-ENFRIAMIENTO
[PASS] T07: Prediccion activa pre-enfriamiento
EMERGENCIA: Zona 'test' - temperatura extrema
Clasificacion: Zona 'test' -> aire REGULAR

```



```

[PASS] T08: Emergencia tiene prioridad
Clasificacion: Zona 'test' -> aire BUENO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> MODO AHORRO (vacía)
[PASS] T09: Modo ahorro en zona vacía
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> OPTIMO
[PASS] T10: Umbral exacto: 19.0C es optimo
[PASS] T11: Sin sensores de temp no hace nada raro
Clasificacion: Zona 'zona_B' -> aire REGULAR
Clasificacion: Zona 'zona_B' -> CALUROSO
Clasificacion: Zona 'zona_A' -> aire REGULAR
Clasificacion: Zona 'zona_A' -> FRIO
Control: Zona 'zona_A' -> activar CALEFACCION
Control: Zona 'zona_B' -> activar REFRIGERACION
[PASS] T12: Dos zonas funcionan independientes

```

=====

```

Resultado: 12/12 tests pasados
PASS - T01: Temp baja activa calefaccion
PASS - T02: Temp alta activa refrigeracion
PASS - T03: CO2 alto activa ventilacion
PASS - T04: Condiciones optimas, sin accion HVAC
PASS - T05: Inferencia multinivel: sensor -> confort -> accion
PASS - T06: Riesgo energetico reduce intensidad
PASS - T07: Prediccion activa pre-enfriamiento
PASS - T08: Emergencia tiene prioridad
PASS - T09: Modo ahorro en zona vacía
PASS - T10: Umbral exacto: 19.0C es optimo
PASS - T11: Sin sensores de temp no hace nada raro
PASS - T12: Dos zonas funcionan independientes

```

1.9 Conclusiones

El sistema funciona bien en todos los escenarios que he probado. Lo que mas me ha costado entender es que el orden de las reglas importa mucho, sin el saliente las reglas de control se ejecutaban antes que las de clasificacion y no funcionaba.

Tambien tuve problemas con las acciones duplicadas hasta que use NOT() para comprobar que no existiera ya la accion antes de crearla.

La parte de series temporales esta bien porque permite que el sistema actue de forma preventiva en vez de solo reactiva. Solo predice temperatura pero se podria hacer lo mismo con el CO2 o la ocupacion.